Sri Lanka Institute of Information Technology



IE3132 Penetration Testing

Y3.S1.WE.CS.02.01

Group 44

Assignment 1

Operation Silent Ledger

IT Number	Name
IT24102747	Kosgollage S R
IT24201195	Palugaswewa K I K I B
IT24102899	Shaheen M F M
IT24102514	Aaqib H M

Contents

Executive Summary..... 1

Objectives..... 2

CTF Concept & Scenario..... 3

Selected domains..... 5

Difficulty Model..... 6

Stage specifications..... 7

Platform / Box Architecture..... 11

Technology Stack..... 13

Deployment & Isolation plan..... 14

Security Controls..... 17

Testing / Validation plan..... 18

Individual contributions..... 19

Risks / Limitations..... 20

Conclusion..... 22

References..... 23

Appendices..... 24

Executive Summary

This report introduces the initial-stage design for Operation Silent Ledger; a Capture the Flag (CTF) Play Box being developed as part of the IE3132 Penetration Testing module. This Play Box is constructed as a story-driven CTF revolving around the protagonist assuming the role of an operative in a newly-formed Cyber Command unit, investigating an insider that is believed to be responsible for aiding a number of cyberattacks on neighbouring countries, which had granted an external threat actor access to their networks by providing them with credentials. Under the guidance of the ever-present narrator, HANDLER, the player must complete six stages, each of which provides evidence that aids the investigation, eventually leading to the successful prosecution of the insider responsible.

The design incorporates six domains; OSINT/Reconnaissance, Networking, Web Technologies, Digital Forensics, Cryptography, and Linux/System Security, which were chosen for their applicability to a realistic red team assessment, covering the penetration testing fields from initial intelligence gathering to low-level exploitation of weaknesses to compromise a system. Within the report, each of the CTF stages are detailed, including their scenario, learning outcome, tools required, mechanics and flag logic, and an alternative solution for the more advanced stages.

The design proposed envisions a hybrid architecture, where Docker containers host light service CTF challenges, which can be reset quickly, a dedicated virtual machine is used to host the system-level capstone challenge that requires a realistic assessment environment, requiring analysis of an entire operating system, and static files are distributed as part of the offline analysis CTF stages. The entire infrastructure is managed through a CTFd control panel and is hosted internally on a virtual network to ensure isolation and only exists when a capstone virtual machine is deployed and ready to be interacted with by the player, ensuring technical viability and efficient resource use while still maintaining the immersion and narrative flow of the story by only allowing progression to the next stage once the preceding stage is completed.

Finally, this report details the group's choice of technology stack, deployment and isolation strategies, testing and validation strategies, individual contributions, and an assessment of this design's risks and limitations. As the project is at the early stage, no elements have been built or deployed yet, and this report hopefully addresses the technical aspects that the group will consider when moving towards the implementation phase.

Objectives

- The project aims to design a technically realistic, immersive and narrative themed CTF box Operation Silent Ledger.
- Select appropriate cybersecurity domains that can be transformed into a structured CTF challenge.
- Six domains: OSINT / Reconnaissance, Networking, Web Technologies, Digital forensics, Cryptography and System / security. The above domains have selected and mapped onto a single investigative narrative where each represents a distinct phase of a real-world insider threat investigation, from suspect identification to finding evidence on suspicious activities.
- Design a realistic CTF Box containing six stages of progressive difficulty with clear learning outcomes.
- Six stages are structured as a continuous storyline in which each stage outcome directly informs the next. Each has specific learning outcome related to its domain. [2]
- Define the proposed platform, box architecture, services, operating environment and deployment approach.
- A hybrid deployment model which combines Docker container for lightweight, resettable service based challenges with a dedicated virtual machine for system level capstone challenge orchestrated through a CTFd control layer.
- Demonstrate a knowledge of practical cybersecurity concepts through challenge design.
- Each stage is based on real world industry techniques and documented industry practices like OSINT correlation methods, OWASP catalogued web vulnerabilities [5] , standard digital forensic techniques, cryptographic implementation failures and Linux privilege escalation patterns which reflect genuine penetration testing methodology rather than artificial puzzles.
- Plan technical implementation and testing requirements before the full CTF build.
- A phased implementation plan, testing strategy and risk analysis are proposed to guide the build stage and ensure each challenge is solvable, resettable and properly isolated before deployment.
- Document the contribution and responsibility of each member of the group.
- Responsibilities are divided across platform architecture, challenge design and integration with individual contributions recorded in a dedicated matrix.

CTF Concept & Scenario

CTF name - Operation Silent Ledger

Scenario

Following a series of coordinated cyberattacks on Sri Lanka's neighboring countries, national investigators identify evidence that each intrusion was facilitated by an insider with legitimate access, in each case having provided credentials to an external threat actor nicknamed 'Black Tide'. A newly formed Offensive Cyber Response Unit narrows the list of possible insiders to four people in trusted professional positions. The player takes the role of an operative within this unit tasked with identifying and building evidence against the insider with the aim of supporting their arrest and prosecution.

The investigation is framed as a continuous case file rather than a set of disconnected puzzles: [2] each stage provides a specific piece of evidence or intelligence that directly enables the next stage, and which mimics the nature of a real digital forensics investigation starting from broad public information gathering and moving active interception, system compromise evidence recovery, decryption and finally a system level breach to obtain conclusive proof.

Delivery mechanism

The scenario is delivered through a persistent narrator persona of a 'Handler' - a cyber command case officer communicating with the participant through typed, CLI - style dispatches before and after each stage. Handler provides context to the mission, reveals the outcome of each completed stage and raises the stakes of the narrative as the investigation progresses. This is implemented as a briefing layer within the CTF platform, displayed on challenge unlock and updated on successful flag submission, providing the player with a consistent sense of progression and consequence throughout the six stages.

Target audience

Undergraduate level cybersecurity students with a foundational knowledge across networking, web security and operating systems, consistent with the intended difficulty progression defined for the six stages. The challenges assume familiarity with basic command line tools and common security utilities but not advanced or specialist expertise, making it an appropriate introductory to intermediate practical assessment or training exercise.

Suspect framework

4 suspects are introduced in the mission briefing who are all plausible and in positions of legitimate access.

Subject A – Senior network administrator, National Telecommunications Authority

Subject B – Independent security consultant with active government contracts

Subject C – IT Director at a private financial clearing firm with government linked accounts

Subject D – University cybersecurity lecturer, formerly employed by a state security agency

Rather than eliminating one suspect per stage in a rigid one-to-one fashion however, the suspect list narrows in two phases. Stages 1-2 reduce the field from four suspects to one primary suspect through the process of public record correlation and confirmed active communication with Black Tide. Stages 3-6 then shift focus from identification to evidence building systematically constructing a case good.

enough to support prosecution. This two-phase structure reflects proper investigative practice, where identifying a suspect and proving a case are distinct tasks with different techniques.

Flag Format

All flags follow a consistent format reinforcing the investigation theme:

CASE {stage_keyword}

Each flag is prefixed with CASE{and suffixed with }, framing every discovered flag as a logged piece of evidence within the Agent's case file, consistent with the narrative's framing of the participant building a prosecutable case. The keyword inside the braces is unique to each stage and reflects the specific evidence recovered (e.g., CASE{recon_confirmed} for Stage 1, CASE{root_ledger_confirmed} for Stage 6). This format is validated centrally through CTFd's static string comparison for all six stages, regardless of the underlying challenge backend (static file, container, or VM).

Learning Outcomes Supported by the Scenario

The scenario is designed so that narrative progression and technical skill development occur together: participants must correctly apply reconnaissance, network analysis, web exploitation, forensic recovery, cryptanalysis, and system exploitation techniques in a logical sequence, with each stage's evidence serving as a prerequisite for the next. This reinforces LO1 (information gathering), LO2 (evaluating appropriate tools/techniques per scenario), and LO3 (developing exploitation approaches) as defined in the assignment's learning outcomes, while maintaining narrative and technical continuity throughout

Selected domains

Domain	Stage	Justification
OSINT / Reconnaissance	1	Chosen as the entry point since actual investigations start from the publicly available information. It introduces participants to the correlation-based analysis (metadata review, username tracing, image geolocation) without the actual involvement in the infrastructure operation, which is a fair starting point without being over-difficult. [3]
Networking	2	Chosen to introduce active interception and traffic analysis, expanding on the identification work of Stage 1. It exposes participants to protocol-level inspection and anomaly detection, core competencies in assessing live network-based threats. [4]
Web Technologies / Web Security	3	Selected because internal web applications represent common entry points into otherwise well-protected organizations. This domain serves to illustrate established vulnerability classes (e.g., OWASP-catalogued issues). [5] and transitions the narrative from passive observation to active system engagement
Digital Forensics	4	Chosen to reflect the investigative reality that access alone is insufficient. Evidence must be recovered and preserved. This domain introduces file recovery, metadata analysis, and hidden-data detection, skills directly relevant to post-compromise investigation. [6]
Cryptography	5	Selected to test analytical rather than purely tool-based skills, requiring participants to identify and exploit weaknesses in a flawed encryption implementation. This domain was placed after Forensics because the evidence recovered in Stage 4 is only usable once decrypted, reinforcing a logical technical dependency.
Linux/ system security	6	Chosen as the capstone domain because system-level compromise represents the highest-stakes and most technically demanding phase of the investigation. It integrates skills from earlier stages (web-based initial access, careful enumeration) with privilege escalation techniques, providing a suitably challenging conclusion to the investigation.

Difficulty Model

Operation Silent Ledger follows a progressing difficulty curve with its six stages: Easy, Easy–Moderate, Moderate, Moderate–Hard, and Hard. In line with the assignment's expected Easy/Moderate/Hard progression mode

Stage	Difficulty	Purpose
1	Easy	Introduce the scenario and establish baseline investigative technique
2	Easy - Moderate	Introducing active analysis and protocol level investigation
3	Moderate	Require exploitation and application of a specific vulnerability class
4	Moderate	Combine recovery technique with investigative reasoning
5	Moderate - Hard	Require analytical, non tool based problem solving
6	Hard	Integrate skills from all prior stages into a single capstone system compromise

Stages 2-6 define an alternate, distinct solution path to allow difficulty scaling for experienced participants without altering the required flag or narrative outcome

Stage specifications

Field	Detail
Stage ID & Title	S1 – The Open thread
Domain	OSINT / Reconnaissance
Difficulty	Easy – Entry point stage which does not require specific tools. The flag must be found through multi – source correlation rather than a single lookup
Scenario	Follow the mission briefing, the participant is given public facing material on four suspects and must identify inconsistencies that suggest that one or more are concealing their true activity
Learning objective	Apply open-source intelligence correlation techniques (metadata analysis, identity linking, Image geolocation) [3] to build a verifiable lead from disconnected public information
Challenge environment	Static file set, plus a minimal static web hosting setup (e.g., GitHub Pages or a lightweight nginx container) for the mock forum and document pages. No application backend or database required
Player task	Correlate three independent artifacts. PDF with mismatched author metadata, an archived forum post under a matching username and a photo whose background contradicts a claimed alibi to identify C as the primary subject.
Flag location/ logic	Flag string is embedded as a hidden HTML comment in the archived forum post's page source, discoverable only once the participant has traced the metadata username to that specific post
Required tools	Metadata viewer (exiftool), browser developer tools / view source, reverse image search
Dependencies	None (entry stage)
Expected solution path	Extract PDF metadata, identify anomalous username, locate matching forum post, inspect page source for flag, cross check photo geolocation as supporting confirmation
Alternate path	N/A
Validation	Static flag string compared against stored value via CTFd flag checking
Hints	Hint 1:" Documents remember more than their authors intend". Hint 2: "Old accounts rarely delete themselves, they are just forgotten" (Hint 1: -10% of stage points; Hint 2: -20% of stage points, cumulative)
Reset / Recovery	Static content: No reset required. Files/pages are permanently available

Field	Detail
Stage ID & Title	S2 -The Intercept
Domain	Networking
Difficulty	Easy–Moderate -core solution accessible with basic packet analysis; harder alternate path available via TLS fingerprinting
Scenario	A short authorized intercept of traffic suspected to include BLACK TIDE relay infrastructure. The player must extract a codename and timing information of an active contact with Subject C.
Learning objective	Use of protocol-level traffic analysis to identify and decode exfiltrated data.
Challenge environment	A pre-generated .pcap file (static distribution) or a two-container live Docker network (for the more technically minded)
Player task	Isolate anomalous DNS-tunneled traffic within the capture, reconstitute and decode the exfiltrated data to obtain a codename and a time window of contact.
Flag location/ logic	Flag is a CASE {...}string embedded within the decoded DNS payload itself once all fragments are reassembled and decoded (CASE{codename_relay_confirmed})
Required tools	Wireshark/tshark, some knowledge of scripting (Python) to re-assemble and decode the DNS subdomain.
Dependencies	The identification of Subject C as the primary suspect from Stage 1.
Expected solution path	Filtering the capture to obtain anomalous DNS queries [4] , extracting individual encoded subdomains, re-assembling them in chronological order and decoding the resulting string.
Validation	Static flag string compared via CTFd
Hints	1: "Not all DNS queries are asking where something is." 2: "Order matters -check the timestamps."(With point penalties of -10% stage points for hint 1 and -20% for hint 2)
Alternate path	Correlating TLS handshake JA3/JA3S fingerprints across different connections to identify a reused client and the timing of the connection.
Reset / Recovery	Static pcap; if the live version was used, docker-compose down -v && docker-compose up -d will reset the containers.

Field	Detail
Stage ID & Title	S3 - The Front Door Left Unlocked
Domain	Web technologies / Web security
Difficulty	Moderate. Involves the identification and exploitation of a particular category of bugs (IDOR) instead of a generic lookup; requires a certain understanding of the application logic and tampering with requests, which is more complex than a purely passive analysis
Scenario	Subject C’s employer hosts an internally accessible web portal. The participant needs to find a vulnerability that would allow them to gain access to Subject C’s private messages and identify one of the personal devices mentioned there.
Learning objective	Learn to recognize and exploit an Insecure Direct Object Reference [5] to gain access to restricted data.
Challenge environment	Dockerized vulnerable web application (Flask)
Player task	From a low-privilege account, alter an easily predictable message/conversation indicator in the portal's URL in order to access Subject C's private internal messages without having his credentials.
Flag location/ logic	The flag is embedded within the content of the accessed private message.
Required tools	Browser, Burp Suite / intercepting proxy (optional but recommended)
Dependencies	Confirmation of Subject C's status as the active suspect from Stage 2
Expected solution path	Log in/register as low-privilege account → observe message URL → increment or alter the identifier in the URL → access unauthorized conversation → obtain flag.
Validation	Static flag string compared via CTFd
Hints	Hint 1: "Not everything that looks private is actually protected." Hint 2: "IDs that count upward are IDs that can be guessed."(Hint 1: −10% of stage points; Hint 2: −20% of stage points, cumulative)
Alternate path	The identifier is encoded in some fashion by a static, discoverable encoding/key, which must be decoded first before attempting to increment.
Reset / Recovery	Container reset via **docker-compose down -v && docker-compose up -d**, restoring the application and database to initial seeded state.

Field	Detail
Stage ID & Title	S4 - What Wiping Doesn't Erase
Domain	Digital forensics
Difficulty	Moderate. Combines a technical recovery skill (file carving, metadata inspection) with investigatory reasoning to distinguish relevant recovered data from noise; requires tool proficiency beyond passive viewing
Scenario	A forensic image of Subject C's seized personal device is given. Several files were deliberately deleted and a folder was hidden shortly before seizure - not a full-volume wipe - leaving behind recoverable traces in unallocated space and filesystem metadata; the participant must recover a hidden encrypted archive.
Learning objective	Apply file carving and filesystem metadata analysis [6] techniques to recover deleted or concealed data.
Challenge environment	Static disk image file (to be distributed for offline analysis)
Player task	Carve unallocated space for file fragments, examine filesystem metadata and an NTFS Alternate Data Stream to locate and recover a hidden encrypted archive.
Flag location/ logic	Recovering the archive constitutes completion of the stage; the archive contains a text file with the flag in CASE{...} format (e.g.,CASE{ledger_recovered}), submitted once extracted
Required tools	Autopsy or FTK Imager (cross-platform, for image loading and carving); Windows participants may additionally use dir /r to inspect Alternate Data Streams directly, while Linux/macOS participants can use Autopsy's built-in ADS detection or icat/istat from The Sleuth Toolkit for equivalent inspection
Dependencies	Requires the device reference obtained in Stage 3
Expected solution path	Mount the disk image, then use any carving utility to recover unallocated space. Further inspection of the recovered files' metadata or any alternate data streams (ADS) will reveal a hidden archive. The flag is the hash/name of the recovered archive file.
Validation	The flag was validated against the recovered file's known hash/name by the CTFd service.
Hints	Hint 1: "Quick format doesn't mean quick deletion." Hint 2: "Some data lives in a file's shadow, not the file itself." (Hint 1: −10% of stage points; Hint 2: −20% of stage points, cumulative)
Alternate path	Some data was hidden using LSB steganography inside a cached image file, instead of an ADS. This would have required using some steganalysis tools to recover.
Reset / Recovery	Static file; a fresh copy of the disk image will be re-uploaded if it was somehow corrupted during analysis.

Field	Detail
Stage ID & Title	S5 - The flaw in the lock
Domain	Cryptography
Difficulty	Moderate–Hard — requires analytical cryptanalysis instead of simply brute-force or dictionary attacks; participants need to reason about the ciphertext structure and use statistical methods (index-of-coincidence, frequency analysis) to find the key, as there is no obvious “correct” tool to use
Scenario	The archive obtained in Stage 4 is encrypted with a home-built algorithm. The player needs to break the encryption to find direct evidence of collaboration with BLACK TIDE.
Learning objective	Recognize and exploit weaknesses in real-world cryptographic schemes.
Challenge environment	Static encrypted file (comes with a decryption stub, no live server required)
Player task	Determine that the encryption uses repeating-key XOR, find the key length with the index-of-coincidence method [7] , recover the key with frequency analysis, and decrypt the archive.
Flag location/ logic	The flag is contained in the decrypted message as a phrase that directly implies collaboration with BLACK TIDE.
Required tools	Python (for custom scripts or to run pycryptodome), CyberChef (optional)
Dependencies	Successfully completing Stage 4 to obtain the encrypted archive
Expected solution path	Analyze ciphertext structure → estimate key length → apply frequency analysis per key-length block → recover key → decrypt → locate flag phrase
Validation	CTFd static string comparison
Hints	Hint 1: “A repeated pattern in ciphertext is rarely a coincidence.” Hint 2: “Once you know the key length, treat each block like its own puzzle.” (Penalty: −10% for Hint 1, −20% for Hint 2)
Alternate path	The archive is encrypted with RSA using a very small public exponent and no padding; the player can recover the plaintext by using a low-exponent attack [7]
Reset / Recovery	No dynamic state is stored on the server.

Field	Detail
Stage ID & Title	S6 - The last door
Domain	Linux / System Security
Difficulty	Hard. Capstone stage requiring chained exploitation (initial foothold -> enumeration -> privilege escalation) across a full operating system, combining skills from all previous stages; highest technical and cognitive load in the series
Scenario	Evidence suggests the presence of a personal server managed by Subject C outside of official oversight. The participant is required to breach it in order to retrieve a final piece of court-admissible evidence regarding credential misuse.
Learning objective	Apply enumeration and privilege escalation techniques to compromise a hardened Linux system to find protected evidence
Challenge environment	Dedicated virtual machine (VirtualBox), booted on demand upon completion of Stage 5, network-isolated until unlocked
Player task	Gain initial low-privilege access via a vulnerable exposed service, perform enumeration to find misconfigurations (sudo -l, SUID binaries, cron jobs) to escalate to root and find the final evidence log
Flag location/ logic	Flag is a CASE{...}string embedded within the decrypted message content, explicitly confirming coordination and motive (e.g., CASE {blacktide_motive_confirmed})
Required tools	Requires motive/coordination evidence confirmed in Stage 5; VM only becomes reachable to the network once Stage 5's flag has been submitted
Dependencies	Requires motive/coordination evidence confirmed in Stage 5; VM only becomes network-reachable after Stage 5's flag is submitted
Expected solution path	Exploit the outdated web-facing service for initial foothold -> perform enumeration in search of privilege escalation vectors -> escalate to root -> locate and read evidence log -> submit flag
Validation	Static flag string compared via CTFd
Hints	Hint 1: "Don't guess your way to root. Check what you're already allowed to run.", Hint 2: "Scheduled tasks trust their scripts more than they should." (Hint 1: -10% of stage points, Hint 2: -20% of stage points, cumulative)
Alternate path	Instead of a writable cron script, escalation requires reverse engineering a vulnerable custom SUID binary for a command injection vulnerability
Reset / Recovery	VM reverted to a clean pre-built snapshot via VBoxManage snapshot restore, triggered per-attempt or per-team session

Platform / Box Architecture

Platform and Box Architecture

The Operation Silent Ledger is designed by implementing the core as a control center for challenge access, flag validation, and narrative dissemination, while individual challenges are hosted on either a set of containerized services or a single virtual machine, depending on their technical requirements.

Architectural Overview

The participants are connected to a single access point, which is in this case the CTFd platform serving as the sole entry point to the challenges, the description of which, the submission of flags, and the progress tracking are handled. However, behind this control layer, the challenges are deployed on two different backends according to their needs:

- **Docker containers**, used for Stage 2 (Networking) and Stage 3 (Web Technologies), provide lightweight, quickly resettable environments suited to service-based challenges.
- **A dedicated virtual machine**, used exclusively for Stage 6 (Linux/System Security), providing a realistic operating system environment necessary for genuine privilege escalation testing.
- **Static file distribution**, used for Stage 1 (OSINT), Stage 4 (Forensics), and Stage 5 (Cryptography), requiring no live infrastructure as these stages involve offline analysis of provided artifacts.

All live infrastructure (containers and the VM) are on an isolated internal network, and cannot reach the public internet or the host they run on, as to prevent exploitation from affecting the institutional or external network. The VM is turned off and isolated from the network until stage 5 is complete, at which point an automatic trigger turns it off of a clean snapshot and gives the participant network access to it, in order to save resources while keeping the "final door" progression.

A narration/briefing layer is implemented alongside the control layer, where HANDLER's dispatches precede and follow each step. This is realized by the use of CTFd's challenge description fields in combination with a webhook-based update mechanism which exposes the subsequent part of the narrative upon successful flag submission, thus keeping the story in sync with the technical progress.

Flag validation is unified for all stages in CTFd, be it from a static file, a container or the VM, submission and validation takes place in the same way from the user's perspective.[1]

Diagram Description

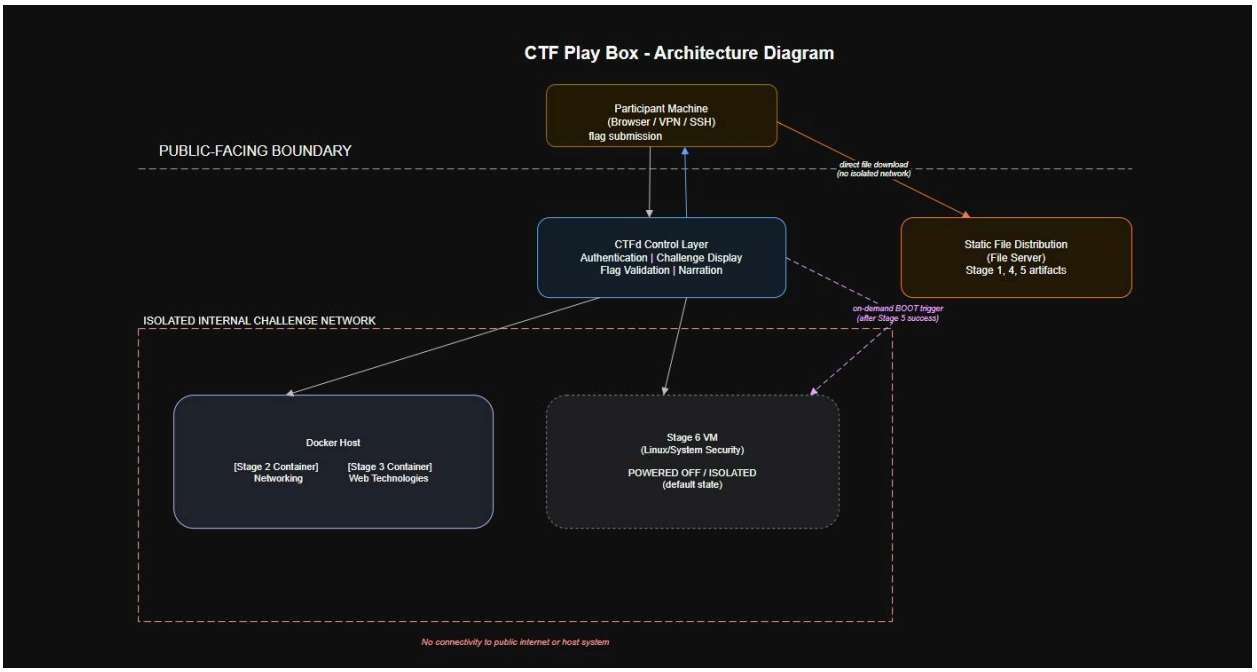


Figure 1.Architecture Diagram

Estimated resources

Resources	Requirement
CPU	4-core for the host server (CTFd + Docker). Additional 2-core for the Stage 6 VM when active
RAM	8 GB for CTFd and Docker containers; 4 GB for the stage 6 VM during its active period
Storage	20 GB total: CTFd platform (2 GB), Docker images (3 GB), Stage 6 VM image and snapshot (10 GB), static artifacts including disk image, encrypted archive, pcap, documents (5 GB)
Network	One isolated internal network (private subnet) for Docker host and VM host; one externally reachable interface for the CTFd control layer only
Software	Ubuntu server 22.04 LTS (host OS), Docker Engine + Docker compose, VirtualBox 7.x, CTFd, Python 3.x with required libraries (Scapy, pycryptodome)

These estimates are based on relatively small scale, since concurrent use by multiple teams would require proportionally larger allocation of CPU/RAM resources as mentioned in the Risks/ Limitations section for repetitive execution of stage 6.

Ports and security

Service	Port	Protocol	Access
CTFd	443	TCP	Public facing, participant access point
Stage 3 vulnerable web app	5000	TCP	Internal only, reverse-proxied through CTFd or accessed via VPN
Stage 6 VM -SSH	22	TCP	Internal only, reachable only after Stage 5 completion
VPN Gateway (Wireguard)	51820	UDP	Public facing, required for participants to reach internal challenge network
Orchestration Webhook Listener	8080	TCP	Internal only, CTFd – webhook script communication

Technology Stack

Category	Technology	Justification
CTF Control layer	CTFd	Open-source, purpose-built for handling issues, flag validation, scoring, and narrative/description delivery; widely used in academic and competitive CTF settings [1], reducing the need for custom platform development
Containerization	Docker + Docker compose	Provides lightweight, fast-resetting isolated environments for Stage 2 and Stage 3; Compose allows multi-container setups (e.g., the Stage 2 network simulation) to be setup and deleted with a single configuration file [8]
Virtualization	VirtualBox	Free, scriptable via VBoxManage command-line control, and supports snapshot-based reset, necessary for Stage 6's realistic OS-level privilege escalation environment, which containers cannot fully replicate [9]
Web Application (Stage 3)	Flask (Python)	Lightweight framework allowing precise control over the deliberate IDOR vulnerability without unnecessary framework complexity that could obscure the intended flaw
Network Traffic generation	Scapy (Python)	Enables scripted, repeatable generation of realistic DNS-tunneled traffic for the intercepted. pcap artifact
Forensic Image creation	FTK Imager/ dd	Industry-standard tools for creating forensically sound disk images, ensuring the artifact reflects realistic forensic recovery conditions
Cryptographic challenge generation	Python (pycryptodome)	Used to construct the deliberately flawed repeating-key XOR (and alternate RSA) encryption scheme in a controlled, reproducible way [7]
Privilege escalation target	Ubuntu server (Linux)	Widely used, well-documented distribution allowing realistic misconfiguration scenarios (SUID, sudo, cron) consistent with real-world system administration practices
Player side analysis tools	Wireshark/ tshark, exiftool, Autopsy, Burpsuite, linpeas	Standard, freely available tools expected to be used by participants; listed for completeness of the technical environment expectations
Orchestration	Custom webhook script (Python/ bash)	Bridges CTFd's flag-correct event to VBoxManage commands, enabling the on-demand VM boot and network unlock upon Stage 5 completion

Deployment & Isolation plan

Deployment Method

The platform follows a hybrid deployment model, combining containerized services, a dedicated virtual machine, and static file distribution, all coordinated through the CTFd control layer:

- **CTFd** is deployed via its official Docker Compose configuration on a dedicated host server, serving as the single point of access for participants.
- **Stage 2 and Stage 3** are deplodockeryed as Docker containers on the same host server as CTFd, isolated via Docker's internal bridge network.
- **Stage 6** is deployed as a pre-built VirtualBox virtual machine image, configured once with the intended vulnerabilities and preserved as a clean snapshot for repeated resets.
- **Stages 1, 4, and 5**do not require any live deployment; their artifacts (documents, disk image, encrypted archive) are hosted as static downloadable files, either through CTFd's file attachment feature or a lightweight separate file server..

In order to prevent participants to download and solve Stages 4, 5 out of order, both of these stages are configured in CTFd to have the previous stage (Stage 3 for Stage 4, Stage 4 for Stage 5) as a requirement (prerequisite) to be visible and/or attempted. This mechanism is similar to the one already in place for Stage 6, which is only revealed upon solving Stage 5. As such, all six Stages will be revealed in order regardless of their underlying implementation (static files, container, or VM)

Implementation Timeline

Phase	Activities	Owner	Milestone
Environment setup	Deploy CTFd, configure Docker host, prepare base VM image	Kosgollage S R	Platform reachable, base infrastructure running
Static Stage build	Build Stage 1, 4, 5 artifacts (documents, disk image, encrypted archive)	Palugaswewa K I K I B , Shaheen M F M	All static artifacts created and checksummed
Live stage build	Build Stage 2 pcap generation, Stage 3 web app, Stage 6 VM configuration	Palugaswewa K I K I B , Shaheen M F M	All containers/VM functional and resettable
Integration	Connect orchestration webhook, configure CTFd gating/requirements, wire narration layer	Kosgollage S R, Aaquib H M	End-to-end flag flow works across all 6 stages
Testing and validation	Execute full testing plan (solvability, reset, isolation, load)	Aaqib H M (coordinating all members)	Validation checklist fully signed off

Network Design and Isolation

All live infrastructure (the Docker host and the VM host) are on a dedicated isolated internal network (networking with Docker containers is done on the same host as CTFd's own network interface through internal bridge/host only networking), even though they are running on the same physical host.

- **Docker internal/bridge networks**, ensures containers can communicate with each other (where required, e.g., for the Stage 2 simulated traffic) but cannot reach the public internet or unrelated services. [8]
- **VirtualBox internal networking**, keeping the Stage 6 VM's network interface confined to the isolated challenge network rather than bridged to the host or public network.[9]
- **Controlled orchestration access**, where CTFd only communicates with the isolated network via a defined orchestration API/trigger mechanism (not direct access), thereby limiting the isolated network's exposure to a single auditable control path.

This design guarantees that the exploitation opportunities that will be used by the participants, including the privilege escalation on the Stage 6 virtual machine, will not cause damage to institutional systems, the systems of the other participants, or any other systems outside of the specific challenge's scope, thus fulfilling the requirement of the assignment, which states that the proposed environment should ensure that the exploitation opportunities will not cause unintended damage to institutional or public systems.

On-Demand VM Activation

The Stage 6 VM is powered off and network-isolated for the duration of Stages 1-5, and only activated/powered on when the flag is submitted correctly (successfully completes Stage 5). This is done via a webhook that is triggered by a CTFd's flag-correct event, which does a VBoxManage snapshot [VM name] restore [snapshot name] to restore it to a clean state, then a VBoxManage startvm [VM name] --type headless to allow it to be network-reachable in the isolated segment.

The network reachability is achieved by attaching the VM's network interface to the isolated challenge network, with VBoxManage modifyvm [VM name] --nic1 intnet --intnet1 [challenge-network-name] as part of the same webhook script, rather than a firewall rule change. Making sure the VM is genuinely unreachable (no interface on the shared network) rather than just having its packets filtered until this point.

The reason this on-demand approach was chosen over keeping the machine up and running at all times is because of:

1. **Resource efficiency** - the VM is idle most of the time (for a typical playthrough), thus saving the CPU/RAM resources of the host machine it's running on, which is

important if more than one participant or team is playing through the CTF stages concurrently

2. **Narrative reinforcement** - Making the VM reachable only after solving Stage 5 emphasizes the feeling of it being a "final door", as opposed to being freely accessible from the beginning

The compromise is that there's a delay between when the player finishes Stage 5 and when they can access Stage 6 (because the VM needs to be booted up first), but this is seen as an acceptable trade-off for the benefits mentioned, and is clearly stated in the Risks/Limitations section

Reset and Recovery Mechanisms

- **Docker-based stages (2, 3)**:It can be reloaded using the docker-compose down && docker-compose up command to reinitialize the container to its original seeded state; can be reloaded on demand or on a schedule.
- **VM-based stage (6)**:It can be reloaded using VBoxManage snapshot restore [9] command to restore the machine to its original clean image before testing or on demand.Stages based on static files
- **Static file stages (1, 4, 5)**: These stages do not need reloading since a participant's input cannot alter the contents of the distributed files. However, a clean version of the files is stored in case one of the distributed files becomes corrupted or needs to be resent.

Security Controls

Control	Implementation
Isolation	Docker internal networks and VirtualBox internal networking restrict all live infrastructure to a single internal network with no route to the public internet or host system
Reset mechanism	Docker containers are reset via docker-compose down -v && docker-compose up -d. The stage 6 VM is reset via VBoxManage snapshot restore. Static artifacts are inherently unmodifiable and checksummed for integrity
Privilege restrictions	Participants will interact only with low privilege accounts by default. Privilege escalation is a deliberate, contained objective within Stage 6 only.
Logging	CTFd records all flag submissions attempts, timestamps and challenge access per participant. The on-demand VM trigger events are logged separately to confirm the Stage 6 Vm is not reachable before Stage 5 completion
Controlled orchestration access	CTFd communicates with the isolated network only via a defined orchestration API/webhook trigger, rather than direct unrestricted access.

Testing / Validation plan

This section describes how to check that each stage is working properly, can be reset, and is isolated before the platform is open to participants in the later implementation phase.

Solvability Testing

- Each stage will be solved end-to-end by a team member who did not design that specific stage, verifying that the problem is solvable using only the intended tools and techniques.
- The expected solution path documented in each stage specification will be followed exactly to verify that no steps are missing, ambiguous, or reliant on unprovided artifacts/environment.
- Alternate (more difficult) solution paths, where defined (Stages 2,3,4,5,6), will be tested separately to verify that they independently lead to the same flag.

Flag Validation Testing

- Each flag will be verified against CTFd's stored value to confirm correct matching, with attention to common submission errors (case sensitivity, whitespace, formatting inconsistencies).
- Flags embedded inside recovered files, decrypted content, or page source will be tested to confirm that they can indeed be reliably extracted using the mentioned tools, and not accidentally modified during file transfers, hosting, or during container re-building.

Reset and Recovery Testing

- Docker-based stages will be tested by running a repeatable series of "docker-compose down -v && docker-compose up -d" commands to ensure that the container always reverts to its original vulnerable state without any previous data.
- Stage 6 VM will be tested through repeated snapshot-restore cycles to confirm privilege escalation vectors, and the evidence file remains consistent and unmodified after each reset.
- Static file artifacts will be checksummed (e.g., SHA-256) after upload to confirm integrity, with backups retained separately in case reissuing is required.

Isolation and Security Testing

- Network isolation will be verified by attempting outbound connections from within the Docker host and VM host to confirm no route to the public internet or host system exists.
- The on-demand VM trigger mechanism will be tested to confirm the VM remains genuinely unreachable before Stage 5 completion, and becomes reachable **only after** a valid flag submission, and not because of incorrect submission.

- Basic penetration testing of the CTFd deployment itself (default credentials, exposed admin panels, outdated dependencies) will be performed to ensure the control layer is not itself a weak point.

Load and Concurrency Considerations

- If a number of participants or teams is expected to make progress at the same time, allocations of memory and the computer’s processing power to containers and VMs will be tested by simulating simultaneous access to check if the host server can handle multiple instances without any performance issues, especially given how the on-demand VM boot mechanism is supposed to function.

Validation Sign-Off

Each stage will be marked 'validated' only upon passing solvability testing, flag validation, reset testing and isolation testing, with results recorded on a simple validation checklist (Stage ID, tester, date, pass/fail by criterion) to be incorporated as supporting evidence in the report's appendices.

Individual contributions

Member	Role	Contribution	Evidence
Kosgollage S R IT24102747	Platform and architecture	Developed the architecture design, which included the hybrid approach of using the both Docker and VM technologies, the on-demand VM creation process; designed the network isolation architecture; wrote the Platform / Box Architecture, Technology Stack, and Deployment & Isolation Plan sections of the documentation	Architecture diagram, platform specification table, deployment plan section, technology justification write-up
Palugaswewa K I K I B IT24201195	Challenge Design A	Developed full specifications for Stages 1–3 (OSINT, Networking, Web Technologies), including scenario writing, flag logic design, required tools, and alternate harder solution paths for Stages 2 and 3	Stage 1–3 specification tables, contributed artifact concepts (mock forum post, PDF metadata design, IDOR vulnerability design)
Shaheen M F M IT24102899	Challenge Design B	Finalized specifications for Stages 4-6 (Digital Forensics, Cryptography, Linux/System Security), including scenario writing, flag logic design, required tools, and alternate harder solution pathways for all 3 stages; contributed technical justification for the XOR/RSA cryptographic weaknesses and Linux privilege escalation techniques used in the capstone stage.	Stage 4–6 specification tables, contributed technical design for the XOR/RSA cryptographic challenge and Linux privilege escalation scenario
Aaqib H M IT24102514	Integration, testing and documentation	Authored the Testing/Validation Plan and Risks/Limitations section; compiled and formatted the complete report using all members' sections; cross-referenced specifications from each stage with the architecture diagram and the platform description to ensure consistency; managed the compilation of the list of references and appendices, including the AI usage prompt log..	Testing plan document, compiled final report, validation checklist template, appendix compilation

Risks / Limitations

Technical Risks

On-demand VM activation delay
The Stage 6 VM has an initial boot-up period following stage 5 completion, requiring a short wait before it becomes accessible as a capstone experience. It was accepted as a reasonable compromise for the resource efficiency but is identified as a usability limitation that should be measured and potentially optimized (possibly using a pre-warmed but network-isolated VM rather than a fully cold boot) during the practical implementation phase.

Container-to-container network simulation fidelity
It seems that the Stage 2 networking challenge was built using scripted traffic (using Scapy) and not actual communications between adversarial infrastructure to avoid the noise and unpredictability of real traffic. While it is a good exercise to test one's ability to perform protocol layer analysis, this is a slight oversimplification of how traffic is actually generated in the real world. I think the reason for this design choice was to make sure the challenge would be solvable even after multiple resets.

Concurrent access resource contention
In case if multiple participants or teams reach the platform, especially on Stage 6, the host server's CPU/RAM to run multiple VM instances would have not been load tested at this design moment. Which makes it a scalability risk to address during the implementation phase, with possible solutions linked to a VM instance queue or per team schedule.

Security Risks

Scope creep beyond the isolated network
While network isolation is built into the architecture, it may be accidentally turned off during the implementation (e.g., a bridged VMware network adapter, overly permissive Docker network settings), etc., leaving the challenge infrastructure vulnerable to attack. This risk is mitigated by the isolation testing in the Testing/Validation Plan, but remains a concern for any real-world implementation.

Participant privilege escalation beyond intended scope
There is a slight risk that a participant could figure out an unintended privilege escalation vector on the Stage 6 VM other than the one that was built into the challenge, potentially allowing them to compromise the VM in an unexpected way. Re-imaging the VM from a clean snapshot after each session will reduce the impact of this risk.

Design Limitations

Fixed suspect/solution path
The current design fixes Subject C as the confirmed insider, with one primary correct narrative path (plus alternate technical solve methods for each stage, not alternate narrative outcomes) limiting replayability for participants who complete the CTF more than once, though it was considered an acceptable scope limitation for an initial single-playthrough educational design.

Domain coverage tradeoffs
Steganography and Reverse Engineering are not core domains of this design, though both are present (in a limited capacity) in the alternate (more challenging) solution paths that are defined for Stage 4 and Stage 6 respectively. This was done by design, as a way to offer them as optional, higher-difficulty alternatives that are not presented as primary stages. This avoids unnecessary overlap with Digital Forensics and Web Technologies while still exposing stronger participants to both domains. Both are viable options for standalone stages in a future expansion beyond the minimum of six required. This is noted here as a deliberate scope decision, not an oversight, with both domains being identified as candidates for further stage expansion beyond the six minimum required.

Design-stage scope
As explicitly permitted by the assignment brief, this submission represents a design and planning milestone only. No component described in this report, including the Docker containers, the VM or the CTFd deployment, have so far been fully built or tested in a live environment. All testing and validation described in this report represents the planned approach to be executed in the subsequent implementation stage, not results already obtained.

Dependency Risks

Sequential stage dependencies
Since each stage's evidence is the basis for the following one (e.g., the recovered archive of Stage 4 is required to complete Stage 5), any implementation error or corrupted artifact in early stages may halt the progress in subsequent ones. This is why static artifact checksumming and backup copies (mentioned in the Deployment & Isolation Plan) are required, but the sequential design is acknowledged to be less fault-tolerant than a completely independent, non-narrative challenge set would be.

Conclusion

This report aims to discuss the design of the Operation Silent Ledger narrative-driven CTF box, which is intended to be a storyline-driven penetration testing challenge based on an

insider-threat investigation. The creation of the six stages, which are organized to gradually increase in difficulty and cover six distinct areas of cybersecurity, was completed according to the requirements of this assignment. This design demonstrates understanding of the required material, as each stage is based on actual penetration testing techniques and real-life incident response practices.

The challenges of the first and second stages were inspired by the actual methods of open-source investigations and vulnerability research, which are used in the respective fields of cybersecurity. The former involves gathering the required information by analyzing publicly available data, as practiced by investigative journalists. In turn, the latter utilizes the OWASP methodology, which covers common vulnerabilities and their respective exploitation techniques. Other stages involve elements of digital forensics, such as data recovery and analysis procedures, historical cryptographic algorithm exploitation scenarios, and privilege elevation assessment practices that are commonly encountered in Linux systems. This approach to the challenge design was selected to emphasize the similarities between the investigative aspects of a penetration test and the goals of the storyline.

The suggested CTF platform design is based on the combination of Docker, a separate virtual machine that is used for on-demand challenges, and static files, which are managed by the CTFd engine with additional isolation measures. Such an approach to platform design is appropriate, as it balances the host system isolation requirements and the ability to implement individual challenges with increased difficulty levels.

Overall, this report does not aim to provide a proof-of-concept prototype but rather a detailed description of the CTF platform, which can be further developed by the group. The primary purpose of this design is to cover all the necessary aspects of the project, including the selected architecture, specific stages, technology stack, technical isolation practices, and a series of testing procedures, which will ensure the functionality of the prototype. In this report, the potential risks and limitations of the selected approach are also reviewed. However, they are intended as guidelines for the implementation stage rather than the design phase, as some additional factors will be taken into consideration when developing the prototype.

By reviewing the requirements and challenges of a penetration test and designing a CTF challenge around it, this report demonstrated the understanding of the different aspects of the penetration testing. In particular, the creation of the CTF box prototype was completed in accordance with the requirements of the assignment, while utilizing the appropriate considerations and practices. Therefore, this report creates a reliable foundation for the development of the complete CTF play box in the later module.

References

[1] CTFd LLC, “CTFd: The easiest capture the flag framework,” *CTFd Documentation*. [Online]. Available: https://docs.ctfd.io/ [Accessed: Sep. 20, 2026].

[2] J. Davis, A. Leek, M. Zhivich, K. Gwinnup, and W. Leonard, “The fun and future of CTF,” in *Proc. 2014 USENIX Summit on Gaming, Games, and Gamification in Security Education (3GSE)*, San Diego, CA, USA, Aug. 2014.

[3] M. Bazzell, *Open Source Intelligence Techniques: Resources for Searching and Analyzing Online Information*, 10th ed. Independently published: IntelTechniques, 2023.

[4] MITRE, “Application Layer Protocol: DNS, Technique T1071.004 – Enterprise,” *MITRE ATT&CK*. [Online]. Available: https://attack.mitre.org/techniques/T1071/004/ . [Accessed: Sep. 20, 2026].

[5] OWASP Foundation, “OWASP Top 10:2021 – A01 Broken Access Control,” *OWASP*, 2021. [Online]. Available: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ . [Accessed: Sep. 20, 2026].

[6] B. Carrier, *File System Forensic Analysis*. Boston, MA, USA: Addison-Wesley, 2005.

[7] J. Katz and Y. Lindell, *Introduction to Modern Cryptography*, 3rd ed. Boca Raton, FL, USA: CRC Press, 2020.

[8] Docker Inc., “Networking overview,” *Docker Docs*. [Online]. Available: https://docs.docker.com/engine/network/ . [Accessed: Sep. 20, 2026].

[9] Oracle Corporation, “Oracle VM VirtualBox User Manual: Virtual networking and snapshots,” *VirtualBox Documentation*. [Online]. Available: https://www.virtualbox.org/manual/ . [Accessed: Sep. 20, 2026].

Appendices

Appendix A - AI Usage prompt Log

In accordance to the assignment's usage policy, a generative AI tool (Claude) was used to refine the investigatory storyline, draft stage narrative text and used to support scenario developmental aspects relevant to the selected domains. All technical design decisions, stage specifications and architecture choices were reviewed and approved by the group prior inclusion into this report.

A summarized log of key prompts used in provided below

Summarized prompt	Purpose
Give me a full summary on the requirements	Understanding assignment scope
Suggest selected domains to stage mapping and narrative sequencing	Initial scope development
Based on the plot I have entered. Generate a script on how the entire drama plays out	Structuring the 6-stage progression while maintaining the storyline
Write a long form short version of the CTF plot with alternate solutions	Producing the full narrative script
Integration of Docker, VM and CTFd into a seamless CTF playbox	Understanding platform architecture, tools and technologies needed to setup stages
Organize the references in IEEE format	Compiling the references list
Review the compiled document against assignment requirements	Checking whether the document meets the assignment requirements

Appendix B - Validation checklist table (Sample)

Stage ID	Tester	Date	Solvability	Flag validation	Reset test	Isolation test	Pass/fail
S1							
S2							
S3							
S4							
S5							
S6							